



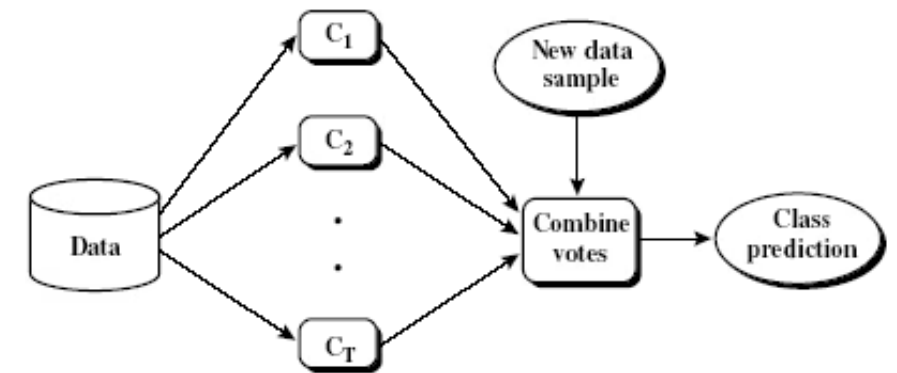
Ensemble Learning in R: Bagging, Random Forest, and Boosting Explained

Ensemble Methods: Boosting Accuracy

Ensemble methods combine multiple learning models to produce predictions that are more robust and accurate than any single model. This approach leverages diversity among classifiers, simultaneously reducing variance and bias.

The core idea is that a group of "weak learners", when properly combined, can outperform a single "strong learner". Each model captures different aspects of the data, and their aggregation yields more reliable results.

Popular techniques include bagging, boosting, and random forests — each with distinct characteristics in terms of model aggregation and base model construction. The following slides explore each in detail.



Reference: J. Han, J. Pei, and H. Tong, Data Mining: Concepts and Techniques, 4th edition. Cambridge, MA: Morgan Kaufmann, 2022.

Bagging: Bootstrap Aggregation

Bootstrap Sampling

Multiple training subsets are created by sampling with replacement from the original dataset. Each subset is the same size as the original, but contains different combinations of instances.

Parallel Training

An independent model is trained on each bootstrap subset. Models are built in parallel with no interaction between them during training, ensuring full independence.

Prediction Aggregation

Predictions from all models are combined through majority voting (classification) or averaging (regression) to produce the ensemble's final output.

Bagging is particularly effective at reducing the variance of high-complexity models, such as deep decision trees, making them more stable and less prone to overfitting — a natural lead-in to Random Forest.

Bagging in Practice: cla_bagging

```
# Loading the package
library(daltoolbox)

# Data and levels
iris <- datasets::iris
slevels <- levels(iris$Species)

# Train/test split
set.seed(1)
sr <- sample_random()
sr <- train_test(sr, iris)
iris_train <- sr$train
iris_test <- sr$test

# Training
# nbagg: number of bootstrap aggregations
model <- cla_bagging("Species", nbagg = 25)
model <- fit(model, iris_train)
```

```
# Evaluation — train
train_pred <- predict(model, iris_train)
train_eval <- evaluate(model,
  adjust_class_label(iris_train[, "Species"]),
  train_pred)
# accuracy: (train result)

# Evaluation — test
test_pred <- predict(model, iris_test)
test_eval <- evaluate(model,
  adjust_class_label(iris_test[, "Species"]),
  test_pred)
# accuracy: (test result)
```

Key Features of cla_bagging()

- Use `cla_bagging()` with `nbagg` (number of bootstrap aggregations)
- `nbagg=25`: 25 independent trees trained on bootstrap samples
- Requires the `ipred` package (`install.packages("ipred")`)
- Aggregates predictions via majority voting (classification)
- Reduces variance by averaging over diverse bootstrap models

📄 Bagging (Bootstrap Aggregation) trains multiple independent models on resampled data and combines their predictions. `cla_bagging()` wraps `ipred::bagging`, providing a clean `daltoolbox`-compatible interface with `fit()`, `predict()`, and `evaluate()`.

Random Forest

Random Forest extends Bagging by introducing an additional layer of randomness known as the **random subspace method**: at each node split, only a random subset of features is considered as split candidates — not all features. This deliberate feature randomness **reduces correlation between individual trees**, which in turn reduces the variance of the ensemble. Each tree is grown to its full depth with no pruning, keeping individual models as strong (low-bias) learners. The final prediction is determined by **majority voting** (classification) or **averaging** (regression) across all trees.

Construction Methods

Forest-RI

Random Input Selection: At each node, F features are randomly selected from all input features as split candidates. The best split among those F candidates is chosen. Trees are grown using CART to maximum depth, with no pruning.

Forest-RC

Random Linear Combinations: Instead of selecting existing features, new synthetic features are constructed as random linear combinations of existing ones. This further decorrelates trees and can capture more complex boundaries.

OOB Evaluation

Out-of-Bag Error Estimation: Each bootstrap sample uses ~63.2% of training data; the remaining ~36.8% (OOB samples) are used to evaluate each tree. Aggregating OOB predictions provides a built-in, unbiased estimate of generalization error — no separate validation set required.

- ❏ **Advantages:** Insensitive to the number of features selected per split and to **irrelevant features** in the dataset. Provides built-in **feature importance estimation** by measuring how much each feature contributes to reducing impurity across trees. Naturally **robust to overfitting** as the ensemble averages out individual tree errors. Generally faster and more scalable than traditional boosting methods.

References: L. Breiman, "Random Forests," *Machine Learning*, vol. 45, pp. 5–32, 2001. Also: J. Han, J. Pei, and H. Tong, *Data Mining: Concepts and Techniques*, 4th ed. Morgan Kaufmann, 2022.

Random Forest in Practice: cla_rf

```
# Loading the package
library(daltoolbox)

# Data and levels
iris <- datasets::iris
slevels <- levels(iris$Species)

# Train/test split
set.seed(1)
sr <- sample_random()
sr <- train_test(sr, iris)
iris_train <- sr$train
iris_test <- sr$test

# Training
# mtry: features per split
# ntree: number of trees
model <- cla_rf("Species", slevels, mtry=3, ntree=5)
model <- fit(model, iris_train)
```

```
# Evaluation — train
train_pred <- predict(model, iris_train)
train_eval <- evaluate(model,
  adjust_class_label(iris_train[, "Species"]),
  train_pred)
# accuracy: 0.975

# Evaluation — test
test_pred <- predict(model, iris_test)
test_eval <- evaluate(model,
  adjust_class_label(iris_test[, "Species"]),
  test_pred)
# accuracy: 0.967
```

Step-by-step

- Use `cla_rf()` with `mtry` (features per split) and `ntree` (number of trees)
- `mtry=3`: at each split, 3 features are randomly sampled as candidates
- `ntree=5`: ensemble of 5 trees (larger values typically yield more stable results)
- Training accuracy: **0.975** | Test accuracy: **0.967**
- Robust to heterogeneous features; reduces variance through ensembling

📌 **Key concept:** Random Forest is an **ensemble of trees** trained with randomness in both samples and features. Combining multiple trees reduces overfitting and increases robustness — putting the theory from previous slides into practice.

Boosting

While bagging builds models in parallel independently, boosting takes a sequential approach: each new model focuses on correcting the errors made by its predecessors, progressively refining the ensemble's accuracy.

Initial Model

Train the first weak classifier with uniform weights assigned to all instances.

Weight Adjustment

Increase weights for misclassified instances; decrease weights for correctly classified ones.

Iteration

Train a new model focusing on the harder instances; repeat the process until the stopping criterion is met.

Final Combination

Aggregate all models using weights proportional to their individual accuracy.

Popular algorithms include AdaBoost, Gradient Boosting, and XGBoost. Boosting is especially effective at reducing bias, though it may be more susceptible to overfitting if not properly regularized.

Boosting in Practice: cla_boosting

```
// Loading the package
library(daltoolbox)

// Data and levels
iris <- datasets::iris
slevels <- levels(iris$Species)

// Train/test split
set.seed(1)
sr <- sample_random()
sr <- train_test(sr, iris)
iris_train <- sr$train
iris_test <- sr$test

// Training
// mfinal: number of boosting iterations
model <- cla_boosting("Species", mfinal = 50)
model <- fit(model, iris_train)
```

```
// Evaluation — train
train_pred <- predict(model, iris_train)
train_eval <- evaluate(model,
  adjust_class_label(iris_train[, "Species"]),
  train_pred)
// accuracy: (train result)

// Evaluation — test
test_pred <- predict(model, iris_test)
test_eval <- evaluate(model,
  adjust_class_label(iris_test[, "Species"]),
  test_pred)
// accuracy: (test result)
```

Step-by-step

- Use `cla_boosting()` with `mfinal` (number of boosting iterations)
- `mfinal=50`: 50 sequential weak classifiers, each correcting the previous
- Requires the `adabag` package (`install.packages("adabag")`)
- Implements AdaBoost: misclassified instances receive higher weights in each round
- Final prediction is a weighted vote across all classifiers

❏ **Key concept:** `cla_boosting()` wraps `adabag::boosting`, implementing the AdaBoost algorithm. Unlike Bagging, models are built sequentially — each one focuses on the hardest instances from the previous round, progressively reducing bias.

XGBoost: Extreme Gradient Boosting

XGBoost is a powerful, regularized gradient boosting framework. Unlike AdaBoost, which reweights instances, XGBoost fits each new tree to the residual errors (gradients) of the loss function. It leverages second-order Taylor expansion for optimization and incorporates L1 and L2 regularization directly into the objective function to control model complexity and prevent overfitting.

It introduces several key innovations over classic Gradient Boosting, including built-in regularization (λ , α), column subsampling (similar to Random Forest), sparsity-aware split finding for handling missing values, and parallel tree construction for increased efficiency.

Gradient Fitting

Each tree fits pseudo-residuals from the loss gradient, using second-order Taylor expansion for optimized objective function.

Regularization

L1 (Lasso) and L2 (Ridge) penalties on leaf weights directly reduce overfitting by controlling model complexity.

Advanced Parameters

Key parameters include `eta` (learning rate), `max_depth` (tree depth), `subsample` (row sampling), and `colsample_bytree` (column sampling).

XGBoost is widely adopted in machine learning competitions and industrial applications due to its remarkable speed, scalability, and predictive accuracy, often outperforming other ensemble methods.

Reference: T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," *KDD 2016*, pp. 785–794. ACM.

XGBoost in Practice: cla_xgboost

```
# Loading the package
library(daltoolbox)

# Data and levels
iris <- datasets::iris
slevels <- levels(iris$Species)

# Train/test split
set.seed(1)
sr <- sample_random()
sr <- train_test(sr, iris)
iris_train <- sr$train
iris_test <- sr$test

# Training
# nrounds: number of boosting rounds
# params: list of xgboost parameters
model <- cla_xgboost("Species", nrounds = 20)
model <- fit(model, iris_train)
```

```
# Evaluation — train
train_pred <- predict(model, iris_train)
train_eval <- evaluate(model,
  adjust_class_label(iris_train[, "Species"]),
  train_pred)
# accuracy: (train result)

# Evaluation — test
test_pred <- predict(model, iris_test)
test_eval <- evaluate(model,
  adjust_class_label(iris_test[, "Species"]),
  test_pred)
# accuracy: (test result)
```

Step-by-step

- Use `cla_xgboost()` with `nrounds` (boosting rounds) and optional `params` list
- `nrounds=20`: 20 gradient boosting trees fitted sequentially to residuals
- Requires the `xgboost` package (`install.packages("xgboost")`)
- Advanced tuning via `params`: e.g., `list(eta=0.1, max_depth=6, subsample=0.8)`
- Automatically handles multiclass via `multi:softprob` objective

📌 **Key concept:** `cla_xgboost()` wraps the `xgboost` package, implementing Gradient Boosting with built-in L1/L2 regularization. It is one of the most competitive ensemble methods in practice, combining speed, scalability, and high accuracy.

Acknowledgements



Data Analytics Lab

dal.global@grupo.cefet-rj.br | <https://eic.cefet-rj.br/dal>

